One's Complement

Negating = invert each bit from $0 \rightarrow 1$ & $1 \rightarrow 0$

$= 2^n - x - 1$

Two's Complement

Negating = invert each bit from $0 \rightarrow 1$ & $1 \rightarrow 0$ and then add 1 to result.

Overflow

Adding two 32-bit numbers can yield a result that needs 33-bit to fully expressed,

Lack of 33rd bit ⇒ when overflow occurs, sign bit is set with value of the result instead of sign of result

4 Cases

1. Add two "+" numbers
2. Add two "−" numbers
3. subtract a "−" from "+" number
4. subtract a "+" from "−" number

Slide 6

* No overflow of adding a positive and a negative number.

(overflow term is misleading ⇒ not mean a carry "overflowed")

```
  0000 0111   (+7)
+ 1111 1010   (-6)
-------------
  0000 0001 = (+1)
```

not overflow ←

```
  0111 1111   (+127)
+ 0000 0010   (+2)
-------------
  1000 0001 ≠ (+129)
```

↑ negative?

8-bit representation

Multiplication

Steps:

- Take digit of multiplier one at a time from right to left
- Multiply the multiplicand by the single digit of multiplier
- Shift intermediate product one digit to left of earlier intermediate products

n-bit mcand & m-bit multiplier ⇒ product in (n+m) bits

Each step of multiplication

- Place a copy of mcand in the proper place if the multipler digit is 1.
- Place ϕ in the proper place if digit is ϕ

First version

Multiplier in 32-bits
Mcand & product in 64-bits
Move multiplicand left one digit at each step to be added with intermediate products.
Multipler is shifted right at each step.

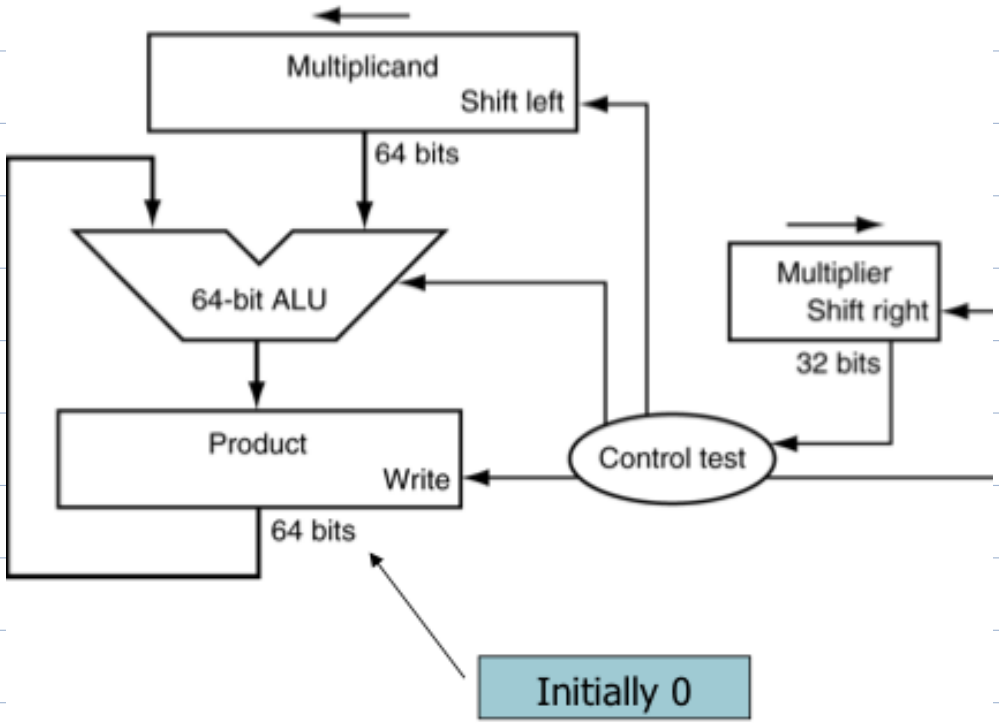
Control decide when to shift mcand & multiplier & when to write

Example: mcand 0010 multiplier 0011

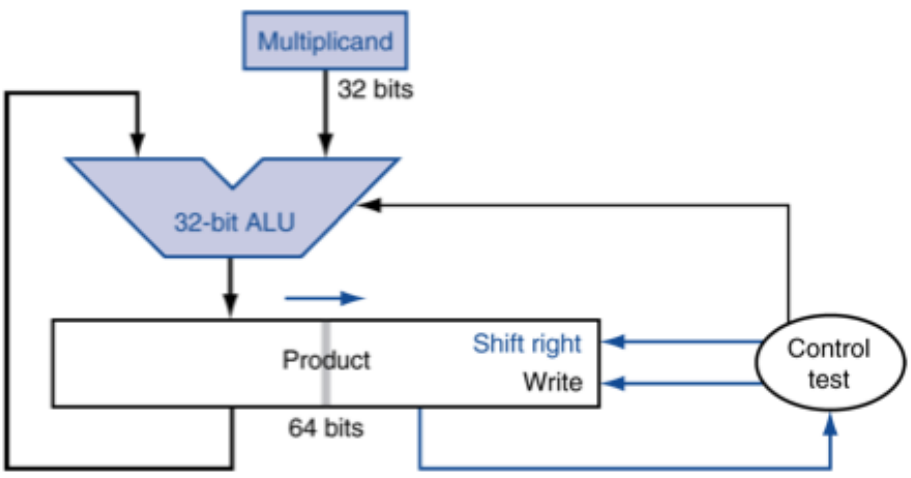
Iteration ↑

I	Step	Multiplier	Mcand	Product
0	initial values	0011	0000 0010	0000 0000
1	prod = prod + Mcand Mcand ← Multiplier →	0001	0000 0100	0000 0010
2	prod = prod + mcand mcand ← multiplier →	0000	0000 1000	0000 0110
3	mcand ← multiplier →	0000	0001 0000	0000 0110
4	mcand ← multiplier →	0000	0010 0000	0000 0110

Multiplicator Hardware



Optimized Multiplier



Problems

(1) 3 steps repeated 32 times (for 32-bit)
If each 1cycle → 3 cycles per step
(perform operations in parallel) 3 cycle → 1 cyc

(2) Half of bits in mcand always Ø
(full 64-bit ALU wasteful) ✓
↳ slow for adding Ø

(3) Mcand shifted left → Not affect least significant bits of product.

Solution: Mcand is fixed relative to product & we shift product right

(4) product waste space that match exactly size of multiplier

Solution: Combine right half of product with multiplier.

In parallel: Multiplier & mcand are shifted while Mcand is added to product if multiplier bit is 1.
(Ensure that it tests right bit of multiplier & get preshifted version of mcand)

0010 x 0011 (for optimized version)

Iteration	Step	Multiplicand	Product
0	Initial values	0010	0000 0011
1	1a: 1 => Prod = Prod + Mcand	0010	0010 0011
	2: Shift right Product	0010	0001 0001
2	1a: 1 => Prod = Prod + Mcand	0010	0011 0001
	2: Shift right Product	0010	0001 1000
3	1: 0 => no operation	0010	0001 1000
	2: Shift right Product	0010	0000 1100
4	1: 0 => no operation	0010	0000 1100
	2: Shift right Product	0010	0000 0110